PROJECT DRAFT · ARTIFICIAL INTELLIGENCE COURSE · AUGUST 2026

# SpecX

## Verified Website-to-WebMCP Conversion, Sandboxed Agent Testing and Safety-Preserving Optimisation

SpecX converts developer-authorised websites into verified, machine-readable WebMCP tool interfaces for AI agents. It grounds every generated tool in executable contracts, stress-tests it inside a disposable browser sandbox, and improves it through a statistically gated closed loop — augmented with four learning-based subsystems: adversarial deep-reinforcement-learning fuzzing, graph-neural-network risk prediction over typed workflow graphs, a multimodal patch-generation model, and a streaming anomaly-detection safety layer.

| | |
|---|---|
| Document type | Full project draft — features, literature review, concepts, implementation, publication plan |
| Existing functionalities | F1 Contract-Grounded Verification · F2 Closed-Loop Optimisation · F3 Evidence-Based Generation · F4 Agent-Readiness Evaluation |
| New functionalities | S1 Adversarial DRL Fuzzing · S2 GNN Workflow-Risk Prediction · S4 Multimodal Structured Patching · S6 Streaming Anomaly Detection |
| Deliverables | CLI · CI integration · web dashboard · verified WebMCP definitions · workflow manifests · agent-readiness reports · research paper |
| Target venues | IEEE Access (primary) · Empirical Software Engineering · Journal of Systems and Software |

SECTION 01

# Abstract & Motivation

WebMCP lets web applications expose structured, machine-readable tools to AI agents: a page carrying WebMCP tool definitions effectively behaves as a client-side MCP server that agents can call directly, instead of scraping and clicking through the raw DOM. The proposal, which grew out of the MCP-B project and is now advanced through the W3C Web Machine Learning Community Group by Google and Microsoft, shipped as an early preview behind a flag in Chrome 146 in February 2026, with broader native support expected later in the year. The opportunity is therefore immediate — and so is the risk: auto-generated tools may invoke the wrong action, cause unapproved side effects, silently leak data through background requests, or be so poorly named and described that agents cannot reliably select them.

SpecX addresses this end to end. It (i) analyses a developer-authorised website's repository, DOM, accessibility tree, API traces and existing end-to-end tests to generate atomic WebMCP tool wrappers; (ii) derives executable preconditions and postconditions for every tool and verifies them in a controlled browser sandbox, quarantining anything unsafe, unverifiable or incorrect; (iii) optimises tool names, descriptions and schemas through measured agent testing behind a statistical regression gate that never trades safety for performance; and (iv) evaluates agent readiness across four conditions — raw DOM, generated, verified, and optimised WebMCP — over typed workflow graphs.

Following the review committee's provisional acceptance, this draft incorporates four additional AI functionalities selected from the eight suggested enhancements: S1 an adversarial deep-reinforcement-learning fuzzer that hunts for contract violations, race conditions and hidden side effects; S2 graph neural networks that turn typed workflow graphs into embedding spaces for predicting unmapped data dependencies and high-risk execution paths; S4 a fine-tuned multimodal transformer that converts DOM diffs, network traces and screenshots into structured code patches and schema corrections; and S6 a streaming anomaly-detection layer (Isolation Forest + LSTM autoencoder) that catches slow-leaking unapproved effects during sandboxed testing. The selection was made after a prior-art study (Section 3) to maximise defensible novelty, architectural coherence, one-semester feasibility, and clean ablation design for the accompanying journal paper.

> Thesis of the paper. Contract-verified, adversarially tested and learning-optimised WebMCP interfaces measurably improve agent success, efficiency and safety compared with raw DOM interaction and unverified generated tools — and each added subsystem contributes an independently measurable share of that improvement.

SECTION 02

# Existing Functionalities (F1–F4)

The four functionalities below form the accepted core of SpecX. F1 and F2 were proposed as completely new; F3 and F4 as upgrades of existing generator and benchmark work. Together they define the pipeline into which the new S-features plug.

F1 · NEW

## Contract-Grounded Tool Verification

Derives *executable* preconditions and postconditions for every candidate tool from five evidence surfaces: repository code, rendered DOM, accessibility (ARIA) tree, network traces, and optional backend audit events. Each tool is executed in a controlled browser — a disposable sandbox or an MCP-B Chrome integration — and its observable effects are checked against the contract. Tools with incorrect, unapproved or unverifiable effects are quarantined rather than shipped.

Output: pass/fail verdicts per contract clause, effect diffs, quarantine list with machine-readable reasons.

F2 · NEW

## Safety-Preserving Closed-Loop Optimisation

An external evaluation agent exercises the verified tools on realistic held-out tasks. Failure traces guide LLM-driven revisions of tool *names, descriptions and approved JSON schemas* — the agent-facing metadata that governs tool selection — never the underlying action. A statistical regression gate accepts a revision only when repeated trials show improvement *and* all safety invariants still hold.

Output: optimised tool metadata, per-revision trial statistics, accepted/rejected revision log.

F3 · UPGRADE

## Evidence-Based Website-to-WebMCP Generation

Extends form/action generators with repository parsing, semantic DOM/ARIA signals, API traces and existing end-to-end tests to identify candidate actions and emit implementation-ready, *atomic* WebMCP tool definitions (one verifiable action per tool). Atomicity is what makes F1's contracts checkable.

Output: WebMCP tool definitions with parameter schemas, provenance links back to the evidence that justified each tool.

F4 · UPGRADE

## Comparative Agent-Readiness Evaluation & Workflow Graphs

Extends web-agent benchmarks with four conditions — raw DOM, generated, verified, and optimised WebMCP — so every pipeline stage is measurable. Builds *typed workflow graphs* whose nodes are tools (with type signatures and risk levels) and whose edges encode valid ordering, state transitions and data dependencies.

Output: agent-readiness report per application version, workflow manifests, benchmark scores per condition.

Project outcome. A standalone CLI, CI integration and web dashboard that convert authorised websites into verified, performance-optimised WebMCP interfaces with tests, diagnostics,

workflow manifests and reports — plus a controlled research study of whether contract-verified WebMCP interfaces improve agent success, efficiency and safety.

SECTION 03

# Literature Review & Prior-Art Positioning

A deep prior-art search was conducted across the eight suggested enhancements and the SpecX core (arXiv, ICSE/FSE/ISSTA, NeurIPS/ICML/ICLR, WWW, plus the 2025–2026 MCP/WebMCP ecosystem). Three conclusions drive this draft: the WebMCP problem is real and current; the SpecX core has close 2026 neighbours that must be cited and explicitly distinguished; and the four selected S-features each occupy a defensible gap.

## 3.1 WebMCP and the MCP ecosystem

WebMCP originated in the MCP-B project (Alex Nahas, early 2025) and was unified into a joint Google/Microsoft proposal in August 2025; the W3C Web Machine Learning Community Group adopted it in September 2025 and published a Draft Community Group Report in February 2026. Chrome 146 ships an early preview behind a testing flag. On the security side, static scanners such as MCP-Scan and Cisco's mcp-scanner audit MCP server definitions for prompt-injection and tool-poisoning patterns — but they analyse *declared* tools statically. SpecX differs categorically: it *generates* tools from live websites and verifies their *runtime effects* in a sandbox.

## 3.2 Closest neighbours to the SpecX core (must-cite)

| Work | What it does | How SpecX differs |
|---|---|---|
| ContractSkill (arXiv 2603.20340, 2026) | Compiles web-agent skills into contracted artifacts with preconditions, step specs, postconditions and recovery rules; enables deterministic verification and patch-based repair. Lifts VisualWebArena skill success from ~9–11% to ~28–38%. | Verifies hand-drafted skills against page state only. SpecX targets generated WebMCP tools, grounds contracts in five surfaces (code, DOM, ARIA, network, audit events), and adds a generation stage ContractSkill lacks. |
| CoEvoSkills (arXiv 2604.01687, 2026) | Skill generator and information-isolated surrogate verifier co-evolve through generate–verify–refine cycles; oracle returns opaque pass/fail. | Optimises skill code via an LLM surrogate. SpecX optimises agent-facing metadata (names, descriptions, schemas) behind a statistical regression gate with hard safety invariants. |
| Formal Verification of Agent Skills (arXiv 2605.23951, 2026) | Three verification surfaces; checks every world-state change has a matching audit-log envelope; contrasts itself with plain JSON-Schema validation. | Verifies existing skills formally. SpecX unifies generation + multi-surface runtime verification + adversarial fuzzing + closed-loop optimisation in one system, and evaluates agent readiness comparatively. |

Because these works are so close, the paper's related-work section must lead with them and argue the distinction explicitly; the core's novelty rests on the WebMCP generation target and multi-surface grounding, with the S-features carrying independent contributions.

## 3.3 Prior art per suggested enhancement

| # | Enhancement | Closest prior art | Nov. | Verdict |
|---|---|---|---|---|
| S1 | Adversarial DRL fuzzing of tool contracts | WebRLED (curiosity-driven DRL web GUI testing, arXiv 2504.19237); GENESIS-RL edge-case generation; ARLAS adversarial RL for agent safety (arXiv 2510.05442). All target coverage/crashes or prompt injection — none reward contract violations of agent tools. | 4 / 5 | Adopt |

| # | Enhancement | Closest prior art | Nov. | Verdict |
|---|---|---|---|---|
| S2 | GNNs on typed workflow graphs | FLORA / FLORA-Bench (arXiv 2503.11301): GNNs predict agentic-workflow performance (~0.78 acc., up to 125× speed-up vs LLM judging). Provenance-graph GNNs in security. None predict data dependencies or safety-invariant violations. | 4 / 5 | Adopt |
| S4 | Multimodal transformer → structured patches | SVRepair (2602.06090), GALA (2604.08089), VisualRepair (2607.14075): multimodal automated program repair from screenshots. None repair tool schemas/descriptions or consume network traces as a modality. | 4 / 5 | Adopt |
| S6 | Streaming anomaly detection on traces + DOM mutations | Hybrid LSTM-AE + Isolation Forest is standard in network intrusion detection; trajectory-level agent-safety monitors (TRACE/JANUS). Not applied to DOM-mutation + network streams during tool sandbox verification. | 3.5 / 5 | Adopt |
| S3 | CLIP-style code↔DOM↔trace contrastive embedding | GNN-Coder (2502.15202) already does CLIP-style code–text alignment; Pix2Struct, Screen2Vec cover UI understanding. Requires a large aligned tri-modal corpus + pre-training compute. | 2.5 / 5 | Defer |
| S5 | MAML meta-learning for tool descriptions | System Prompt Optimization with Meta-Learning (2505.09666); DSPy/MIPROv2, OPRO are simpler, stronger baselines reviewers will demand. MAML bilevel optimisation is unstable at student scale. | 2.5 / 5 | Defer |
| S7 | RAG over Cypress/Playwright scripts | Autonomous QA Agent (2601.06034): RAG-grounded Selenium generation, 90% execution success vs 30% baseline; Playwright codegen/MCP. Near-solved engineering pattern. | 2 / 5 | Engineering aid only |
| S8 | SHAP/IG readiness scorecards | AgentSHAP (2512.12597) is explicitly the first framework for explaining tool importance in LLM agents via Monte-Carlo Shapley values — pre-empts the core claim. | 2 / 5 | Reporting feature only |

Benchmark anchoring: ST-WebAgentBench (222 policy-paired tasks; Completion-under-Policy metric, under which evaluated agents complete safely far less often than nominally) supplies the safety dimension; OSWorld-MCP (158 auto-generated MCP tools across 7 applications, tool-vs-GUI ablation) is the nearest tool-condition comparator; WebArena and Mind2Web supply task pools. No prior benchmark spans SpecX's four conditions (raw DOM / generated / verified / optimised), which itself is a contribution.

SECTION 04

# New Functionalities (S1, S2, S4, S6)

Four of the eight reviewer suggestions are adopted. Selection criteria: publishable novelty surviving the prior art above, coherence with F1–F4, one-semester feasibility for a small team, and a clean per-feature ablation. Each card states what the feature does, where it plugs in, and its defensible novelty claim.

S1 · NEW

## Adversarial DRL Contract Fuzzer → plugs into F1

A deep-reinforcement-learning agent (PPO/DQN) treats the F1 sandbox as a Gym-style environment. Its *actions* mutate tool parameters, reorder calls, and fire concurrent invocations; its *state* is the observed DOM + network + audit delta; its *reward* is +1 for each newly triggered contract violation, unapproved side effect or race condition, shaped by a curiosity bonus for novel states. Standard LLM-driven test trails explore the happy path; the adversarial agent is paid to break it.

Novelty claim: to our knowledge, the first adversarial-RL fuzzer whose reward is defined over executable pre/postcondition violations of WebMCP/MCP agent tools (vs. coverage/crash rewards in WebRLED & GUI-testing RL).

S2 · NEW

## GNN Workflow-Risk & Dependency Prediction → plugs into F4 (feeds F1)

F4's typed workflow graphs are embedded with graph neural networks (GCN/GAT via PyTorch Geometric). Two heads: (i) *edge prediction* recovers unmapped data dependencies between tools; (ii) *node/subgraph classification* forecasts high-risk execution paths and non-obvious safety-invariant violations across long tool sequences — before any sandbox run. Training labels come from F1/S1 sandbox verdicts, so the fuzzer literally generates the GNN's training data.

Novelty claim: first GNN predicting safety-invariant violations and data dependencies on typed WebMCP workflow graphs — FLORA-style graph learning repurposed from performance prediction to safety.

S4 · NEW

## Multimodal Structured Patch Generator → plugs into F2

Replaces unstructured textual failure traces in the F2 revision loop with a fine-tuned vision-language model (Qwen-VL/LLaVA-class, LoRA-adapted) that ingests serialised DOM diffs, network trace logs and screenshots *simultaneously* and emits schema-constrained JSON: concrete code patches and tool-schema corrections. Structured output makes revisions machine-applicable and auditable by the regression gate.

Novelty claim: first multimodal (DOM-diff + trace + screenshot) model producing structured WebMCP schema/tool patches — adjacent multimodal APR (SVRepair, GALA, VisualRepair) repairs general program bugs, not agent-tool metadata.

S6 · NEW

### Streaming Anomaly-Detection Safety Layer → plugs into F1

During every sandboxed run, windowed features from browser network traces and DOM MutationObserver streams feed two detectors: an Isolation Forest (scikit-learn) for point anomalies and a small LSTM autoencoder trained on approved traces, whose reconstruction error flags sequence anomalies. This catches what deterministic contracts miss: slow-leaking side effects, delayed background POSTs, silent storage writes, unauthorised third-party calls.

Novelty claim: a real-time learned safety layer over DOM-mutation + network streams during agent-tool sandbox verification — a novel application of a well-proven hybrid detector, positioned as complementary to (not replacing) contract checks.

Why not S3, S5, S7, S8. S3 needs a large aligned code–DOM–trace corpus and contrastive pre-training compute a student team cannot marshal, and GNN-Coder weakens its claim. S5 is pre-empted by meta-learned prompt optimisation and is unstable to train; DSPy/OPRO-style baselines would likely win. S7 is an excellent engineering aid (and will be used internally in F3) but is too incremental to claim. S8 is pre-empted by AgentSHAP; it survives as a dashboard reporting feature, not a contribution.

SECTION 05

# Concepts Explained

Every technical concept the system relies on, in the order it appears in the pipeline. This section doubles as the paper's background section and as onboarding material for new team members.

### MCP & WebMCP

The Model Context Protocol (MCP) is a JSON-RPC-based standard through which AI applications discover and invoke external tools with typed parameter schemas. WebMCP moves the tool surface into the web page itself: a site declares JavaScript-backed tools (name, description, input schema, handler), and a browsing agent calls them directly instead of parsing the DOM. MCP-B is the Chrome-extension lineage of the same idea and serves as one of SpecX's two execution targets.

### Atomic tool wrappers

A tool is *atomic* when it performs exactly one verifiable action (e.g., "add item to cart", never "add item and checkout"). Atomicity is what makes contracts decidable: each tool has a small, checkable effect footprint, and workflows are recovered as graphs over atomic tools rather than baked into monoliths.

### Preconditions & postconditions (design by contract)

A *precondition* states what must hold before a tool may run (user logged in, form field present, cart non-empty); a *postcondition* states what must hold after (order row created, exactly one POST to /api/orders, confirmation dialog visible). SpecX derives both as *executable* checks from code, DOM, ARIA, network traces and audit events — the "contract-grounded" part of F1. A tool that cannot be given a checkable contract is quarantined as unverifiable.

### Sandboxed execution & quarantine

Every candidate tool runs inside a disposable browser environment (seeded database, network interception, no external effects) so that misbehaviour is observable but harmless. Quarantine means the tool is excluded from the shipped WebMCP manifest and reported with the exact contract clause it violated.

### Closed-loop optimisation & regression gates

An evaluation agent attempts held-out tasks with the current tool set; failures produce traces; an LLM (later, S4's model) proposes metadata revisions; the revised set is re-trialled. A *statistical regression gate* (paired trials, significance test, effect size) accepts a revision only if performance improves and no safety invariant regresses — preventing the optimiser from "improving" success by weakening safeguards.

### Deep reinforcement learning & adversarial rewards (S1)

RL trains an agent to maximise cumulative reward through trial and error; *deep* RL uses neural networks (here PPO/DQN) over rich state. An *adversarial reward* pays the agent for making the system under test fail — each new contract violation, race condition or unapproved effect is a point. A curiosity bonus (reward for novel states) prevents the fuzzer from replaying the same crash.

### Race conditions & hidden side effects

A race condition is behaviour that depends on the timing/interleaving of concurrent operations (double-submit creating two orders). A hidden side effect is any state change outside the declared contract (an analytics beacon, a background write). S1 searches for both actively; S6 watches for them passively.

### Graph neural networks & typed embeddings (S2)

GNNs learn vector representations of nodes and edges by iteratively aggregating information from neighbours (GCN: spectral convolutions; GAT: learned attention weights). Embedding SpecX's typed workflow graphs lets the system *predict* missing data-dependency edges (link prediction) and classify paths by violation risk (node/subgraph classification) before spending sandbox time — a learned prioritiser in front of the verifier.

### Multimodal transformers & structured decoding (S4)

A vision-language transformer jointly attends over text tokens (serialised DOM diffs, network logs) and image patches (screenshots). LoRA fine-tuning adapts a pre-trained model cheaply by training small low-rank adapter matrices. *Schema-constrained decoding* forces the output to be valid JSON matching a patch schema, so every revision is machine-applicable.

### Isolation Forests & LSTM autoencoders (S6)

An Isolation Forest isolates anomalies with random splits — anomalous points need fewer splits, giving an anomaly score without labelled attacks. An LSTM autoencoder learns to compress-and-reconstruct *normal* sequences; abnormal sequences reconstruct poorly, and the reconstruction error is the alarm. Together they cover point anomalies and temporal anomalies over network/DOM streams.

### Typed workflow graphs & agent readiness (F4)

Nodes are tools annotated with type signatures and risk levels; edges encode valid ordering, state transitions and data flow. *Agent readiness* is the measured ability of a standard agent to complete tasks against an interface, reported across the four benchmark conditions with success, efficiency (steps/tokens) and safety (policy-violation) metrics.

SECTION 06

# Implementation Plan

The plan follows a strict dependency order — S6 → S1 → S2 → S4 — chosen so that each stage produces the data the next one trains on: S6 needs only approved traces; S1 reuses the sandbox and generates violation examples; S2 trains on S1/F1's verdict labels; S4 trains on failure–patch pairs harvested from F2 runs.

## 6.1 Architecture & stack

| Layer | Components | Technology |
|---|---|---|
| Generation (F3) | Repo parser, DOM/ARIA analyser, API-trace miner, atomic tool emitter; internal RAG over Cypress/Playwright scripts (S7 as engineering aid) | Node.js/TypeScript, Playwright, tree-sitter, LLM API |
| Verification (F1) | Contract deriver, disposable sandbox, effect differ, quarantine manager | Playwright + CDP, Docker sandboxes, network interception (mitmproxy), seeded Postgres |
| S6 safety layer | Feature windower over network + MutationObserver streams; Isolation Forest; LSTM-AE | scikit-learn, PyTorch (small LSTM), streaming via CDP events |
| S1 fuzzer | Gym wrapper around sandbox; parameter/ordering/concurrency action space; violation+curiosity reward | Gymnasium, Stable-Baselines3 (PPO/DQN) |
| S2 graph model | Typed graph builder; GCN/GAT with edge-prediction + risk-classification heads | PyTorch Geometric, NetworkX |
| Optimisation (F2 + S4) | Evaluation agent harness, regression gate, VLM patch generator with JSON-constrained decoding | LLM/VLM APIs, LoRA (PEFT), outlines/jsonschema decoding, SciPy stats |
| Evaluation (F4) | Four-condition benchmark runner, workflow-manifest exporter, readiness reports | WebArena/Mind2Web task pools, ST-WebAgentBench-style safety scoring, OSWorld-MCP comparator |
| Delivery | CLI, CI integration, dashboard | Node CLI, GitHub Actions, React dashboard |

## 6.2 Semester timeline (14 weeks)

| Weeks | Milestone | Exit criterion |
|---|---|---|
| 1–2 | F3 generator + F1 sandbox skeleton on 2 seed apps (one e-commerce, one CRUD/admin) | Atomic tools generated with provenance; sandbox replays a tool run deterministically |
| 3–4 | F1 contracts on all five surfaces; quarantine flow; S6 detectors trained on approved traces | Injected slow side effects detected; false-positive rate on benign tools measured |
| 5–7 | S1 Gym environment + PPO fuzzer; baseline random & coverage-guided fuzzers | DRL beats baselines on violations found per sandbox-hour (else invoke fallback, §6.4) |
| 8–9 | S2 graph builder + GNN heads trained on F1/S1 verdicts | GNN beats rule-based dependency checker on held-out precision/recall |
| 10–11 | F2 loop + regression gate; harvest failure–patch pairs; S4 LoRA fine-tune + constrained decoding | Structured patches apply cleanly; loop converges in fewer iterations than text-trace baseline |

| Weeks | Milestone | Exit criterion |
|---|---|---|
| 12–13 | F4 four-condition benchmark across ≥3 applications; full ablation matrix; statistics | Every S-feature has an isolated effect size with significance test |
| 14 | Paper draft, artifact packaging (code, seeds, manifests), dashboard polish | Reproducibility package runs end-to-end from a clean clone |

## 6.3 Experiment & ablation design

- Conditions: raw DOM → generated (F3) → verified (F1+S1+S6) → optimised (F2+S4), each evaluated by the same frozen agent on the same held-out task pool.
- Per-feature ablations: S1 vs random/coverage fuzzing (violations per sandbox-hour, severity mix); S2 vs rule-based checker and MLP-on-features (dependency P/R, pre-sandbox risk ranking quality); S4 vs unstructured text traces and text-only structured patches (success gain per revision, iterations-to-convergence); S6 on/off with injected slow leaks (detection rate, FPR).
- Metrics: task success, steps and tokens to completion, policy-violation rate (Completion-under-Policy style), quarantine precision, time-to-first-violation.
- Statistics: ≥10 paired trials per condition; paired tests with Holm–Bonferroni correction; report effect sizes and confidence intervals — the F2 regression gate already provides this machinery.

## 6.4 Risk register & fallbacks

- S1 underperforms after ~2 weeks of tuning → fall back to coverage-guided fuzzing with curiosity heuristics, reclassify S1 as engineering, and promote S7 (RAG generation) to the fourth ablatable contribution.
- S2 cannot beat the rule-based checker → demote to interpretability/visualisation aid; consider promoting an S8-style scorecard with explicit AgentSHAP differentiation.
- Sparse S4 training pairs → bootstrap with synthetic failure injection (mutate schemas, break selectors) before harvesting real loop data.
- WebMCP spec drift (Chrome preview evolving) → pin to the February 2026 draft; keep the MCP-B extension path as the stable fallback target.

SECTION 07

# Path to a Successful Journal Publication

Acceptance will not hinge on the ML methods themselves — all four are established primitives — but on *honest novelty framing, airtight empirical design, and reproducibility*. The checklist below is what reviewers at the target venues will actually look for.

## 7.1 Contribution list (≥ 4 novel approaches)

- C1 — System. First end-to-end pipeline generating WebMCP tools from developer-authorised live websites with *multi-surface* contract grounding (code + DOM + ARIA + network + audit events) and sandbox quarantine — explicitly differentiated from ContractSkill, CoEvoSkills and formal skill verification.
- C2 — S1. First adversarial-RL fuzzer rewarded on executable contract violations of agent tools.
- C3 — S2. First GNN predicting data dependencies and safety-invariant violations on typed WebMCP workflow graphs (vs. FLORA's performance prediction).
- C4 — S4. First multimodal (DOM-diff + trace + screenshot) model emitting structured WebMCP schema/tool patches inside a safety-gated revision loop.
- C5 — S6. A streaming learned safety layer over DOM-mutation + network streams during tool verification — a novel application, framed as complementary to contracts.
- C6 — Benchmark. The first four-condition agent-readiness benchmark (raw DOM / generated / verified / optimised WebMCP) with typed workflow manifests, released as an artifact.

Every "first" must be hedged as *"to our knowledge"* with a stated literature-cutoff date, and the prior-art search re-run immediately before submission — the field moved fast enough in 2026 that a six-week-old novelty claim can expire.

## 7.2 What determines acceptance

| Determinant | What reviewers expect | SpecX answer |
|---|---|---|
| Novelty defense | Related work engages the closest neighbours head-on, not a survey of distant ones | Dedicated subsection on ContractSkill / CoEvoSkills / arXiv 2605.23951 with a differentiation table; per-feature closest-work citations (WebRLED, FLORA, SVRepair-family, AgentSHAP) |
| Empirical rigour | Controlled conditions, paired trials, significance tests, effect sizes, multiple applications | Four-condition design over ≥3 apps, ≥10 paired trials, Holm–Bonferroni, CIs; the regression gate doubles as the stats engine |
| Ablatability | Each claimed contribution isolated and measured | One ablation arm per S-feature (Section 6.3); no bundled claims |
| Threats to validity | Explicit internal/external/construct threats | Seed-app selection bias, agent-model dependence, WebMCP spec drift, synthetic-injection realism for S6, fuzzer reward hacking for S1 |
| Reproducibility | Public code, data, seeds, exact model versions, one-command replication | Artifact: CLI + seed apps + manifests + trained checkpoints + benchmark harness; pinned Chrome/spec versions |
| Writing & scope | One coherent story, not eight features | Narrative: generate → verify (S6, S1) → predict (S2) → repair (S4) → measure (benchmark) — every feature advances the same safety thesis |

## 7.3 Venue strategy

| Venue | Profile | Fit & strategy |
|---|---|---|
| IEEE Access (primary) | 2025 JIF ≈ 4.2, Q2; broad scope; ~30-day first decision; APC applies; acceptance figures vary by source (~20–45%) | Best first target: values applied novelty + solid experiments; fast enough for a student timeline. Verify current APC and acceptance data before submission. |
| Empirical Software Engineering (Springer) | Q1, CiteScore ≈ 8.6; review often > 12 weeks; demands genuine empirical depth | Choose if the four-condition benchmark study becomes the headline; requires the full statistical apparatus and threats-to-validity treatment |
| Journal of Systems and Software / Automated Software Engineering | JSS 2025 JIF ≈ 3.8 (Q2); AuSE strongly scope-matched to automated generation + verification | Prestige upgrades if early results are strong; AuSE's scope (automation of SE tasks) matches SpecX almost verbatim |
| TOSEM / TSE (aspirational) | Leading SE journals (JIF ≈ 6+); very long cycles; expect major revisions | Only with exceptionally strong results and a mature artifact; not for the course deadline |

## 7.4 Pre-submission checklist

- Re-run the prior-art search (arXiv, Semantic Scholar alerts on "WebMCP", "agent tool verification", "skill contracts") and update every novelty hedge.
- Verify the exact WebMCP draft date against the W3C repository (github.com/webmachinelearning/webmcp) — sources conflict, and a wrong spec date in paragraph one destroys credibility.
- Confirm venue metrics (JIF, APC, acceptance rate) against Clarivate JCR and official journal pages — aggregator numbers disagree.
- Freeze the evaluation agent model and Chrome version; record all seeds; package a one-command replication script.
- Internal red-team review: have a teammate attack each novelty claim with the cited prior work before a reviewer does.
- Secure faculty guidance sign-off (Dr. Yoganand S / CFAIR) on authorship order, ethics of testing only developer-authorised sites, and data release.

> Bottom line. SpecX ships six defensible contributions wrapped in one safety thesis. The core pipeline alone would be judged incremental against 2026 skill-verification work — the four selected S-features and the four-condition benchmark are what make this publishable. Execute in the stated order, keep every claim ablated and hedged, and target IEEE Access first with EMSE as the empirical-track alternative.

APPENDIX

# Key References

Lu et al., “ContractSkill: Repairable Contract-Based Skills for Multimodal Web Agents,” arXiv:2603.20340, 2026.

“CoEvoSkills: Co-Evolving Skill Generation and Surrogate Verification for Web Agents,” arXiv:2604.01687, 2026.

“Methods for Formal Verification of Agent Skills: Three Layers Toward a Mechanically Checkable Capability-Containment Proof,” arXiv:2605.23951, 2026.

“WebRLED: Curiosity-Driven Deep Reinforcement Learning for Web GUI Testing,” arXiv:2504.19237, 2025.

“Adversarial Reinforcement Learning for Large Language Model Agent Safety (ARLAS),” arXiv:2510.05442, 2025.

“FLORA: GNN-Based Performance Prediction for Agentic Workflows / FLORA-Bench,” arXiv:2503.11301, 2025.

“SVRepair,” arXiv:2602.06090; “GALA,” arXiv:2604.08089; “VisualRepair,” arXiv:2607.14075 — multimodal automated program repair, 2026.

“AgentSHAP: Interpreting LLM Agent Tool Importance with Monte Carlo Shapley Value Estimation,” arXiv:2512.12597, 2025.

“GNN-Coder: Boosting Semantic Code Retrieval with Combined GNNs and Transformer,” arXiv:2502.15202, 2025.

“System Prompt Optimization with Meta-Learning,” arXiv:2505.09666, 2025.

“Autonomous QA Agent: RAG-Grounded Test Script Generation,” arXiv:2601.06034, 2026.

“ST-WebAgentBench: Evaluating Safety and Trustworthiness of Web Agents (Completion-under-Policy),” arXiv, 2024/2025.

“OSWorld-MCP: Tool-vs-GUI Evaluation with Auto-Generated MCP Tools,” 2025/2026.

W3C Web Machine Learning CG, “WebMCP,” Draft Community Group Report, Feb 2026 (verify date). github.com/webmachinelearning/webmcp.

Combination of Isolation Forest and LSTM Autoencoder for Anomaly Detection — hybrid detector literature (network intrusion detection).

Note: several 2026 arXiv preprints above may still be under review; confirm final versions, authors and claims before citing them as settled prior art in the submitted manuscript.